

DSA Tutorial - 3

Name: Harsh Sangaonkar
Topic: Sorting Algorithms

Div: SY1-B1

PRN: B25ET1038

INPUT :

```
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void bs(int arr[],int n){
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0)
            break;
    }

    printf("Sorted array using Bubble
sort: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
void ss(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minidx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minidx]) {
                minidx = j;
            }
        }
        if (minidx != i) {
            int temp = arr[i];
            arr[i] = arr[minidx];
            arr[minidx] = temp;
        }
    }
    printf("Sorted array using selection
sort: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void is(int arr[], int n){
    for(int i=1;i<n;i++){
        int key = arr[i];
        int j = i-1;
        while (j>=0 && arr[j]>key){
            arr[j+1] = arr[j];
            j--;
            arr[j+1]= key;
        }
    }
}
```

```

    }
    printf("Sorted array using Insertion
sort: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```

int partition(int arr[],int low,int high){
    int pivot = arr[high];
    int i = low - 1 ;
    for (int j = low;j<high;j++){
        if(arr[j]<pivot){
            i++;
            swap(&arr[i],&arr[j]);
        }
    }
    swap(&arr[i+1],&arr[high]);
    return (i+1);
}

```

```

void qs(int arr[],int l, int high){
    if(l<high){
        int pi = partition(arr,l,high);
        qs(arr,l,pi-1);
        qs(arr,pi+1,high);
    }
}

```

```

void merge(int arr[], int l, int m, int h,
int sa[]) {
    int i = l;
    int j = m + 1;
    int k = l;

```

```

    while (i <= m && j <= h) {

```

```

        if (arr[i] <= arr[j]) {
            sa[k] = arr[i];
            i++;
        } else {
            sa[k] = arr[j];
            j++;
        }
        k++;
    }
}

```

```

while (i <= m) {
    sa[k] = arr[i];
    i++;
    k++;
}

```

```

while (j <= h) {
    sa[k] = arr[j];
    j++;
    k++;
}

```

```

for (int x = l; x <= h; x++) {
    arr[x] = sa[x];
}
}

```

```

void ms(int arr[], int l, int h, int sa[]) {
    if (l < h) {
        int m = l + (h - l) / 2;
        ms(arr, l, m, sa);
        ms(arr, m + 1, h, sa);
        merge(arr, l, m, h, sa);
    }
}

```

```

int main() {
    int n;
    int c;
    printf("Enter Number of elements:
");
    scanf("%d",&n);
    int arr[n];
    printf("Enter Elements of array: ");
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    int sa[n];
    printf("\nChose method: \n1. Bubble
Sort\n2. Selection sort\n3. Insertion
sort\n4. Quick Sort\n5. Merge
Sort\n\n");
    scanf("%d",&c);
    switch (c) {
        case 1:
            bs(arr,n);
            break;
        case 2:
            ss(arr,n);
            break;
        case 3:
            is(arr,n);
            break;
        case 4:
            qs(arr,0,n-1);
            printf("Sorted array using
Quick sort: ");
            for (int i = 0; i < n; i++) {
                printf("%d ", arr[i]);
            }
            printf("\n");

```

```

        break;

        case 5:
            ms(arr,0,n-1,sa);
            printf("Sorted array using Merge
sort: ");
            for (int v = 0; v < n; v++) {
                printf("%d ", arr[v]);
            }
            printf("\n");
            break;

        return 0;
    }
}

```

OUTPUT :

```

Output

Enter Number of elements: 5
Enter Elements of array: 1 32 3 44 5

Chose method:
1. Bubble Sort
2. Selection sort
3. Insertion sort
4. Quick Sort
5. Merge Sort

5
Sorted array using Merge sort: 1 3 5 32 44

=== Code Execution Successful ===

```